

Module M214

Application Cloud Native

Séance 2 : Routage & Mongoose

Express.js + MongoDB

CRUD avec Mongoose

OFPPT

Développement Digital - Option Full Stack

Table des matières

1	Le Système de Routage	2
1.1	Définition	2
1.2	Syntaxe	2
1.3	Méthodes HTTP et CRUD	2
1.4	Les Objets req et res	2
2	Introduction à Mongoose	3
2.1	Qu'est-ce que Mongoose ?	3
2.2	Pourquoi utiliser Mongoose ?	3
2.3	Installation et Connexion	3
3	Le Schéma (Schema)	4
3.1	Qu'est-ce qu'un Schéma ?	4
3.2	Créer un Schéma	4
3.3	Types de données disponibles	4
3.4	Pourquoi définir un Schéma ?	5
4	La Validation	6
4.1	Qu'est-ce que la Validation ?	6
4.2	Pourquoi valider les données ?	6
4.3	Options de validation Mongoose	6
4.4	Exemple de Schéma avec Validation	6
4.5	Messages d'erreur personnalisés	7
5	Le Modèle (Model)	8
5.1	Qu'est-ce qu'un Modèle ?	8
5.2	Créer un Modèle	8
5.3	Schéma vs Modèle	8
5.4	Fichier modèle complet	8
6	Opérations CRUD avec Mongoose	10
6.1	CREATE - Créer un document	10
6.2	READ - Lire les documents	10
6.3	UPDATE - Modifier un document	10
6.4	DELETE - Supprimer un document	11
7	API Complète Express + Mongoose	12
7.1	Structure du projet	12
7.2	models/User.js	12
7.3	index.js - Serveur complet	12
8	Synthèse	14
8.1	Workflow Mongoose	14
8.2	Récapitulatif des concepts	14
8.3	Méthodes Mongoose	14
8.4	Codes HTTP	14

1 Le Système de Routage

1.1 Définition

Routage Express

Le **routage** détermine comment l'application répond à une requête client, défini par :

- Une **URI** : /users, /products/:id
- Une **méthode HTTP** : GET, POST, PUT, DELETE

1.2 Syntaxe

```

1 app.METHOD(PATH, HANDLER);
2
3 // Exemples
4 app.get('/users', (req, res) => { ... }); // Lire
5 app.post('/users', (req, res) => { ... }); // Créer
6 app.put('/users/:id', (req, res) => { ... }); // Modifier
7 app.delete('/users/:id', (req, res) => { ... }); // Supprimer

```

Listing 1 – Structure d'une route

1.3 Méthodes HTTP et CRUD

Méthode	CRUD	Route
GET	Read	/api/users ou /api/users/:id
POST	Create	/api/users
PUT	Update	/api/users/:id
DELETE	Delete	/api/users/:id

1.4 Les Objets req et res

req.params	Paramètres de route (/users/:id)
req.query	Query string (?search=test)
req.body	Corps de la requête (POST, PUT)
res.json()	Envoie une réponse JSON
res.status()	Définit le code HTTP

2 Introduction à Mongoose

2.1 Qu'est-ce que Mongoose ?

Mongoose - ODM

Mongoose est un **ODM** (Object Document Mapper) pour MongoDB et Node.js.

Rôle : Il fait le pont entre votre code JavaScript et la base de données MongoDB. Au lieu d'écrire des requêtes MongoDB brutes, vous travaillez avec des objets JavaScript.

2.2 Pourquoi utiliser Mongoose ?

MongoDB est une base de données **flexible** (schemaless), ce qui signifie qu'on peut insérer n'importe quelles données. Cette flexibilité peut poser des problèmes :

```
1 // Sans validation, on peut inserer n'importe quoi !
2 db.users.insertOne({ nom: "Ahmed", age: 25 });
3 db.users.insertOne({ name: "Fatima", years: "vingt" }); // Erreur
  de champs!
4 db.users.insertOne({ x: 123 }); // Donnees invalides!
```

Listing 2 – Problème sans Mongoose - Données incohérentes

✓ Mongoose résout ces problèmes

- **Structure** : Définit la forme des documents (schéma)
- **Validation** : Vérifie les données avant insertion
- **Typage** : Garantit les types de données (String, Number...)
- **Méthodes** : Fournit des fonctions CRUD simples

2.3 Installation et Connexion

```
1 npm install mongoose

1 const mongoose = require('mongoose');
2
3 // Connexion locale
4 mongoose.connect('mongodb://localhost:27017/mabase')
5   .then(() => console.log('MongoDB connecte'))
6   .catch(err => console.error('Erreur:', err));
7
8 // OU connexion Atlas (cloud)
9 mongoose.connect('mongodb+srv://user:pass@cluster.mongodb.net/
  mabase')
10  .then(() => console.log('MongoDB Atlas connecte'));
```

Listing 3 – Connexion à MongoDB

3 Le Schéma (Schema)

3.1 Qu'est-ce qu'un Schéma ?

Schéma Mongoose

Un **schéma** définit la **structure** des documents dans une collection MongoDB. C'est comme un **plan** ou un **contrat** qui décrit :

- Quels **champs** sont autorisés
- Quel **type** de données pour chaque champ
- Quelles **règles** doivent être respectées (validation)



3.2 Créer un Schéma

```
1 const mongoose = require('mongoose');
2
3 // Définir le schema
4 const userSchema = new mongoose.Schema({
5   nom: String,
6   email: String,
7   age: Number,
8   actif: Boolean,
9   dateInscription: Date
10 });
```

Listing 4 – Définition d'un schéma basique

3.3 Types de données disponibles

Type	Description	Exemple
String	Chaîne de caractères	"Ahmed"
Number	Nombre (entier ou décimal)	25, 19.99
Boolean	Vrai ou faux	true, false
Date	Date et heure	new Date()
Array	Tableau	["a", "b", "c"]
ObjectId	Référence à un autre document	Relation entre collections
Mixed	N'importe quel type	Flexible

3.4 Pourquoi définir un Schéma ?

Avantages du Schéma

1. **Cohérence** : Tous les documents ont la même structure
2. **Documentation** : Le code montre clairement la structure des données
3. **Prévention d'erreurs** : Les fautes de frappe sont détectées

4 La Validation

4.1 Qu'est-ce que la Validation ?

Validation

La **validation** est le processus de vérification des données **avant** leur insertion dans la base de données. Elle garantit que les données respectent certaines règles.

4.2 Pourquoi valider les données ?

⚠ Sans validation - Dangers

- Un utilisateur peut s'inscrire sans email
- Un prix peut être négatif (-100 DH)
- Un âge peut être une chaîne ("vingt-cinq")
- Des champs obligatoires peuvent manquer

La validation protège :

- **L'intégrité** des données (données correctes)
- **La sécurité** de l'application (pas de données malveillantes)
- **La logique métier** (un stock ne peut pas être négatif)

4.3 Options de validation Mongoose

Option	Description	S'applique à
required	Champ obligatoire	Tous types
default	Valeur par défaut	Tous types
unique	Valeur unique dans la collection	Tous types
min	Valeur minimale	Number, Date
max	Valeur maximale	Number, Date
minlength	Longueur minimale	String
maxlength	Longueur maximale	String
enum	Liste de valeurs autorisées	String, Number
trim	Supprimer les espaces	String
lowercase	Convertir en minuscules	String

4.4 Exemple de Schéma avec Validation

```
1 const produitSchema = new mongoose.Schema({  
2   nom: {  
3     type: String,  
4     required: [true, 'Le nom est obligatoire'],
```

```
5      trim: true,
6      minlength: [3, 'Le nom doit avoir au moins 3 caracteres'],
7    },
8    prix: {
9      type: Number,
10     required: [true, 'Le prix est obligatoire'],
11     min: [0, 'Le prix ne peut pas etre negatif']
12   },
13   stock: {
14     type: Number,
15     default: 0,
16     min: 0
17   },
18   categorie: {
19     type: String,
20     required: true,
21     enum: ['Informatique', 'Telephonie', 'Accessoires']
22   },
23   email: {
24     type: String,
25     unique: true,
26     lowercase: true
27   }
28 });
```

Listing 5 – Schéma complet avec validation

4.5 Messages d'erreur personnalisés

```
1 // Syntaxe courte
2 nom: { type: String, required: true }
3
4 // Syntaxe avec message personnalise
5 nom: {
6   type: String,
7   required: [true, 'Le nom du produit est obligatoire']
8 }
9
10 // Validation min/max avec message
11 prix: {
12   type: Number,
13   min: [0, 'Le prix doit etre positif ou zero']
14 }
```

Listing 6 – Syntaxe des messages d'erreur

5 Le Modèle (Model)

5.1 Qu'est-ce qu'un Modèle ?

Modèle Mongoose

Un **modèle** est une classe créée à partir d'un schéma. C'est l'**interface** qui permet d'interagir avec la base de données (créer, lire, modifier, supprimer des documents).

Analogie : Si le schéma est le *plan d'une maison*, le modèle est l'*entreprise de construction* qui construit les maisons selon ce plan.

5.2 Créer un Modèle

```
1 // Syntaxe : mongoose.model('NomModele', schema)
2 const User = mongoose.model('User', userSchema);
3
4 // 'User' = nom du modele (singulier, majuscule)
5 // MongoDB creera automatiquement la collection 'users' (pluriel, minuscule)
```

Listing 7 – Création du modèle

5.3 Schéma vs Modèle

Schéma	Modèle
Définit la structure	Permet les opérations CRUD
Définit les validations	Exécute les validations
Ne communique pas avec MongoDB	Communique avec MongoDB
C'est un "plan"	C'est un "constructeur"

5.4 Fichier modèle complet

```
1 const mongoose = require('mongoose');
2
3 // 1. Définir le SCHEMA (structure + validation)
4 const userSchema = new mongoose.Schema({
5   nom: {
6     type: String,
7     required: [true, 'Le nom est obligatoire'],
8     trim: true
9   },
10  email: {
11    type: String,
12    required: [true, 'L\'email est obligatoire'],
13    unique: true,
14    lowercase: true
15  },
```

```
16   age: {
17     type: Number,
18     min: [0, 'L\'age ne peut pas etre negatif'],
19   },
20   actif: {
21     type: Boolean,
22     default: true
23   }
24 }, {
25   timestamps: true // Ajoute createdAt et updatedAt
26   automatiquement
27 });
28 // 2. Creer le MODELE a partir du schema
29 const User = mongoose.model('User', userSchema);
30
31 // 3. Exporter le modele pour l'utiliser ailleurs
32 module.exports = User;
```

Listing 8 – models/User.js - Fichier modèle complet

6 Opérations CRUD avec Mongoose

6.1 CREATE - Créer un document

```
1 // Methode 1 : Model.create() - Plus concise
2 const user = await User.create({
3   nom: 'Ahmed',
4   email: 'ahmed@mail.com',
5   age: 25
6 });
7
8 // Methode 2 : new Model() + save() - Plus de controle
9 const user = new User({
10   nom: 'Ahmed',
11   email: 'ahmed@mail.com'
12 });
13 await user.save(); // Sauvegarde dans la base
```

Listing 9 – Deux méthodes pour créer

6.2 READ - Lire les documents

```
1 // Tous les documents
2 const users = await User.find();
3
4 // Avec filtre
5 const activeUsers = await User.find({ actif: true });
6
7 // Un seul document par ID
8 const user = await User.findById('507f1f77bcf86cd799439011');
9
10 // Un seul document par critere
11 const user = await User.findOne({ email: 'ahmed@mail.com' });
12
13 // Avec selection de champs, tri et limite
14 const users = await User.find()
15   .select('nom email') // Seulement ces champs
16   .sort({ createdAt: -1 }) // Tri decroissant
17   .limit(10); // Maximum 10 resultats
```

Listing 10 – Méthodes de lecture

6.3 UPDATE - Modifier un document

```
1 const user = await User.findByIdAndUpdate(
2   req.params.id, // ID du document
3   { nom: 'Ahmed Updated', age: 26 }, // Nouvelles valeurs
4   { new: true, runValidators: true } // Options
5 );
```

Listing 11 – Modifier un document

⚠ Options importantes

- `new: true` : Retourne le document **après** modification (par défaut : avant)
- `runValidators: true` : Applique les validations du schéma lors de la mise à jour

6.4 DELETE - Supprimer un document

```
1 const user = await User.findByIdAndDelete(req.params.id);  
2 // Retourne le document supprime (ou null si non trouve)
```

Listing 12 – Supprimer un document

7 API Complète Express + Mongoose

7.1 Structure du projet

```
1 mon-api/  
2   |-- models/  
3   |       |-- User.js           # Schema + Model  
4   |-- index.js                 # Serveur + Routes  
5   |-- package.json
```

7.2 models/User.js

```
1 const mongoose = require('mongoose');  
2  
3 const userSchema = new mongoose.Schema({  
4   nom: { type: String, required: true, trim: true },  
5   email: { type: String, required: true, unique: true },  
6   age: { type: Number, min: 0 }  
7 }, { timestamps: true });  
8  
9 module.exports = mongoose.model('User', userSchema);
```

7.3 index.js - Serveur complet

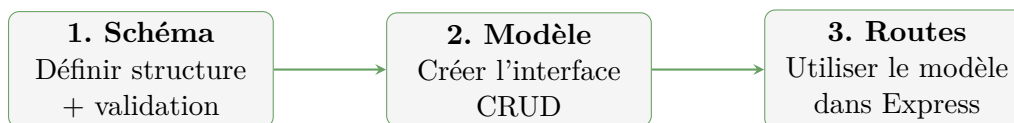
```
1 const express = require('express');  
2 const mongoose = require('mongoose');  
3 const User = require('./models/User');  
4  
5 const app = express();  
6 app.use(express.json());  
7  
8 // Connexion MongoDB  
9 mongoose.connect('mongodb://localhost:27017/mabase')  
10   .then(() => console.log('MongoDB connecte'));  
11  
12 // GET - Tous les users  
13 app.get('/api/users', async (req, res) => {  
14   try {  
15     const users = await User.find();  
16     res.json(users);  
17   } catch (error) {  
18     res.status(500).json({ error: error.message });  
19   }  
20 });  
21  
22 // GET - Un user par ID  
23 app.get('/api/users/:id', async (req, res) => {  
24   try {  
25     const user = await User.findById(req.params.id);  
26     if (!user) return res.status(404).json({ error: 'Non  
    trouve' });
```

```
27     res.json(user);
28   } catch (error) {
29     res.status(500).json({ error: error.message });
30   }
31 });
32
33 // POST - Creer un user
34 app.post('/api/users', async (req, res) => {
35   try {
36     const user = await User.create(req.body);
37     res.status(201).json(user);
38   } catch (error) {
39     res.status(400).json({ error: error.message });
40   }
41 });
42
43 // PUT - Modifier un user
44 app.put('/api/users/:id', async (req, res) => {
45   try {
46     const user = await User.findByIdAndUpdate(
47       req.params.id,
48       req.body,
49       { new: true, runValidators: true }
50     );
51     if (!user) return res.status(404).json({ error: 'Non
52 trouve' });
53     res.json(user);
54   } catch (error) {
55     res.status(400).json({ error: error.message });
56   }
57 });
58
59 // DELETE - Supprimer un user
60 app.delete('/api/users/:id', async (req, res) => {
61   try {
62     const user = await User.findByIdAndDelete(req.params.id);
63     if (!user) return res.status(404).json({ error: 'Non
64 trouve' });
65     res.json({ message: 'Supprime' });
66   } catch (error) {
67     res.status(500).json({ error: error.message });
68   }
69 });
70
71 app.listen(3000, () => console.log('Serveur sur port 3000'));
```

Listing 13 – API CRUD complète

8 Synthèse

8.1 Workflow Mongoose



8.2 Récapitulatif des concepts

Les 3 concepts clés

1. **Schéma** : Définit la *structure* des documents et les *règles de validation*
2. **Validation** : Vérifie que les données sont *correctes* avant insertion
3. **Modèle** : Fournit les *méthodes CRUD* pour manipuler les données

8.3 Méthodes Mongoose

CRUD	Route	Mongoose
Create	POST /api/users	User.create(data)
Read All	GET /api/users	User.find()
Read One	GET /api/users/ :id	User.findById(id)
Update	PUT /api/users/ :id	User.findByIdAndUpdate(id, data, options)
Delete	DELETE /api/users/ :id	User.findByIdAndDelete(id)

8.4 Codes HTTP

Code	Signification	Utilisation
200	OK	GET, PUT réussi
201	Created	POST réussi
400	Bad Request	Validation échouée
404	Not Found	Document non trouvé
500	Server Error	Erreur serveur

✓ Prochaine séance

Séance 3 : Les Middlewares